EXP 12: Implement Iris Flower Classification using KNN

CODING:

```
import math

# Training Dataset
X = [
    [5.1, 3.5, 1.4, 0.2],
    [4.9, 3.0, 1.4, 0.2],
    [5.0, 3.4, 1.5, 0.2],
    [6.4, 3.2, 4.5, 1.5],
    [6.0, 2.9, 4.5, 1.5],
    [5.9, 3.0, 4.2, 1.5],
    [6.5, 3.0, 5.5, 1.8],
    [6.7, 3.1, 5.6, 2.4],
    [6.3, 2.9, 5.6, 1.8]
]

Y = [
    'Setosa', 'Setosa', 'Setosa',
    'Versicolor', 'Versicolor', 'Versicolor',
    'Virginica', 'Virginica', 'Virginica'
]

# Calculate Euclidean Distance
def distance(a, b):
    total = 0
```

Scanned with OKEN Scanner

```
    for i in range(len(a)):
        total += (a[i] - b[i]) ** 2

    return math.sqrt(total)

# KNN Classification
def knn(sample, k=3):
    distances = []

    for i in range(len(X)):
        d = distance(sample, X[i])
        distances.append((d, Y[i]))

    distances.sort(key=lambda x: x[0])

    neighbours = []

    for i in range(k):
        neighbours.append(distances[i][1])

    prediction = max(set(neighbours),
                     key=neighbours.count)

    return prediction

# Display Training Dataset
print("Iris Flower Classification")
print("\nTraining Dataset:")
```

Scanned with OKEN Scanner

```
for i in range(len(X)):
    print(X[i], ":", Y[i])

# Test Samples
test_data = [
    [5.1, 3.5, 1.4, 0.2],
    [6.2, 3.0, 4.5, 1.5],
    [6.5, 3.0, 5.5, 1.8]
]

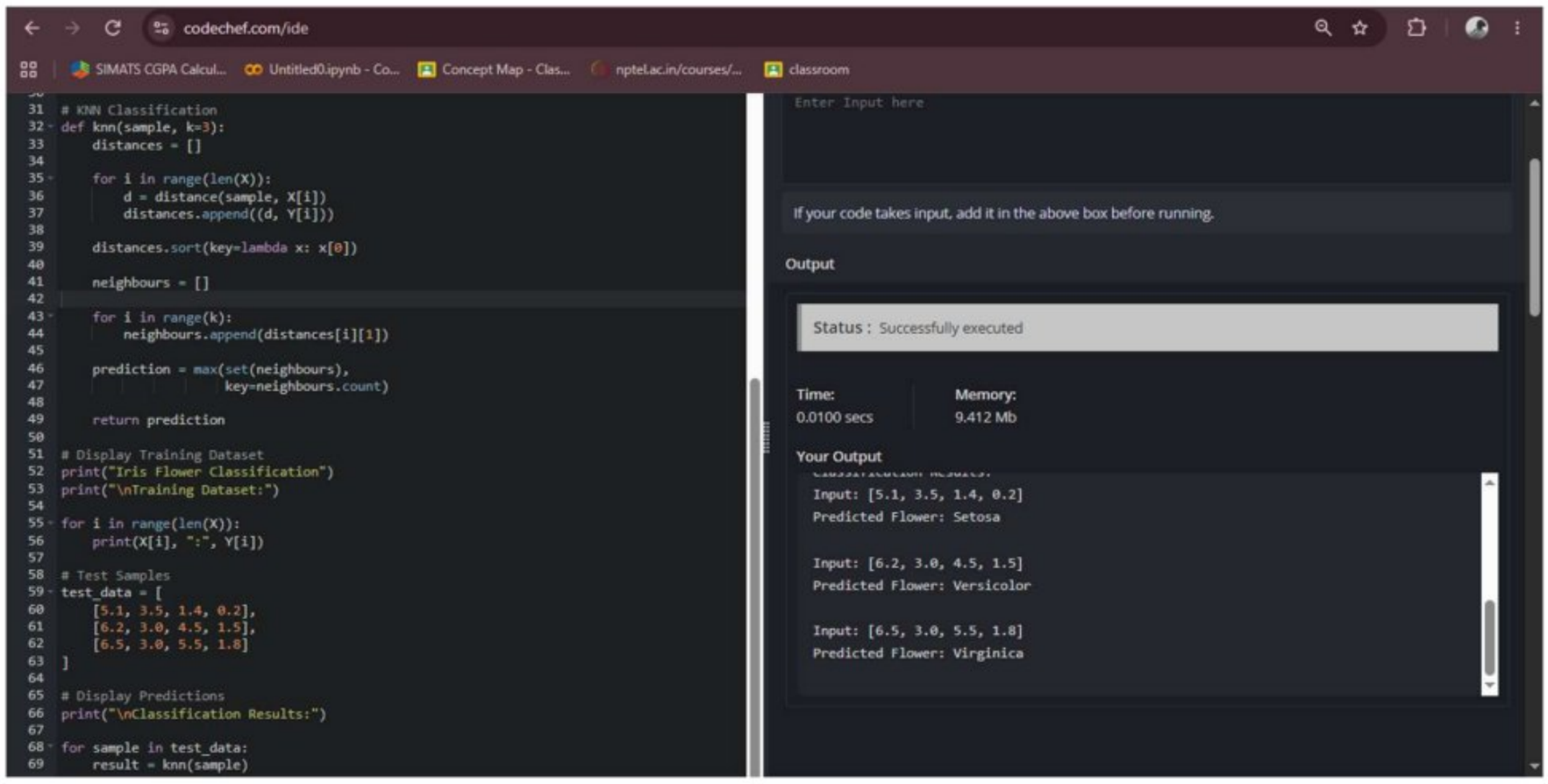
# Display Predictions
print("\nClassification Results:")

for sample in test_data:
    result = knn(sample)

    print("Input:", sample)
    print("Predicted Flower:", result)
    print()
```

Scanned with OKEN Scanner

OUTPUT:



Scanned with OKEN Scanner